SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS: **Total Marks: 50**

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A3- Short Answer Test

Question 1 (5 Marks)
Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.
Question 2 (5 Marks)
Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.
Question 3 (5 Marks)
Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.
Question 4 (5 Marks)
Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.
Question 5 (5 Marks)
Responsive Layout: A college website must display properly on desktops, tablets, and mobile phones. How would you use CSS media queries and flexible layouts to achieve responsiveness?
Question 6(5 Marks)
CSS Styling: An online shopping website has plain product cards. How would you apply CSS colors, borders, shadows, and spacing to improve its visual appearance?
Question 7 (5 Marks)
Navigation Menu: A website navigation menu overlaps its content on smaller screens. How would you use CSS layout techniques to correct the problem?
Question 8 (5 Marks)
JavaScript Validation: A registration form accepts empty fields. How would you use JavaScript to validate the form before submission?

Question 9 (5 Marks)
Interactive Button: An e-learning website requires a button that displays a welcome message when clicked. How would you implement this functionality using JavaScript?
Question 104 (5 Marks)
. Dynamic Content: A shopping website needs to update the total price when a user changes the quantity of a product. How would you use JavaScript to calculate and display the updated price?

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1 (5 Marks)

1. Concept of Responsive Web Design (RWD)

Responsive web design is an approach in which a single web page automatically adapts its layout, images and text to the screen size, resolution and orientation of the device (mobile, tablet or desktop). Instead of building separate sites for each device, one HTML code base is styled with flexible CSS so the content "responds" to the viewport.

2. Importance

- One website and one URL for all devices – lower development and maintenance cost.
- Better user experience: no pinch-zooming or horizontal scrolling on small screens.
- Better SEO – search engines such as Google prefer mobile-friendly sites (mobile-first indexing).
- Wider reach, higher engagement and conversion, and it is future-proof for new screen sizes.

3. Key CSS Techniques for Responsiveness

- **Viewport meta tag:** <meta name="viewport" content="width=device-width, initial-scale=1"> makes the page match the device width.
- **Fluid grids / relative units:** use %, em, rem, vw, vh and fr instead of fixed pixels.
- **Flexible images:** img { max-width:100%; height:auto; } so images shrink with their container.
- **Media queries:** apply different CSS at different breakpoints, e.g. @media (max-width:768px).
- **Mobile-first design:** write base styles for small screens and add min-width queries for larger ones.
- **Modern layout modules:** Flexbox and CSS Grid.

4. Using Flexbox or Grid for Adaptive Layouts

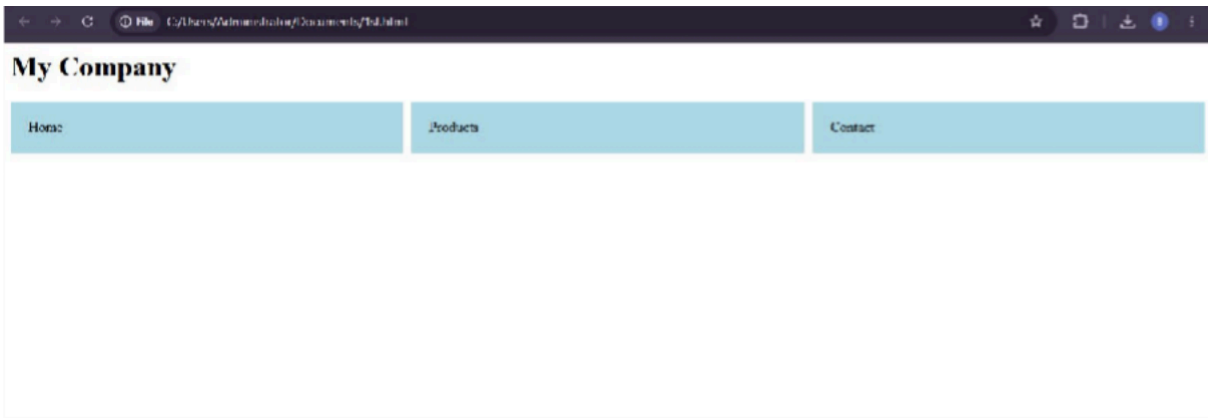
Flexbox is a one-dimensional layout model (row or column). With display:flex, flex-wrap, flex-grow and flex-basis, items share space and wrap automatically onto the next line when the screen is narrow.

CSS Grid is two-dimensional (rows and columns). Using grid-template-columns:repeat(auto-fit, minmax(250px,1fr)) creates as many columns as fit the screen, with no media query required.

5. Example Layout

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
  *{box-sizing:border-box; margin:0;
  padding:0;}
  body{font-family:Arial,sans-serif;}
  header,footer{background:#1f3a5f; color:#fff; padding:16px;
text-align:center;}
```

```
  nav{display:flex; flex-wrap:wrap; gap:12px; justify-content:center;}
  nav a{color:#fff; text-decoration:none; padding:8px 12px;}
  .container{display:grid;
             grid-template-columns:repeat(auto-fit,minmax(250px,1fr));
             gap:16px; padding:16px;}
  .card{background:#f4f6f8; padding:16px; border-radius:8px;}
  .card img{max-width:100%; height:auto;}
  @media (max-width:600px){
    nav{flex-direction:column;
    align-items:center;} body{font-size:16px;}
  }
</style>
</head>
<body>
  <header><h1>My Company</h1>
    <nav><a href="#">Home</a><a href="#">Services</a><a
href="#">Contact</a></nav>
  </header>
  <main class="container">
    <div class="card">Service 1</div>
    <div class="card">Service 2</div>
    <div class="card">Service 3</div>
  </main>
  <footer>&copy; 2026 My Company</footer>
</body>
</html>
```



Result: on a desktop the cards appear in 3 columns, on a tablet in 2 columns and on a mobile phone in a single column, and the navigation stacks vertically below 600 px.

6. Suggested Improvements for User Experience

- Use touch-friendly buttons (minimum about 44×44 px) and adequate spacing.
- Use fluid typography (clamp()) and a readable line length.
- Use responsive images (srcset, <picture>) and lazy loading to speed up mobile loading.
- Replace the menu with a hamburger/collapsible menu on small screens.
- Maintain good colour contrast, use semantic HTML and test on real devices and browser dev-tools.

Question 2 (5 Marks)

1. Client-Side Validation and Its Purpose

Client-side validation is the checking of user input in the browser, using JavaScript (or HTML5 attributes), before the form data is sent to the server. Its purposes are to give instant feedback, reduce unnecessary server requests, save bandwidth, guide the user to correct mistakes and ensure that the data is in the expected format. It improves usability but does not replace server-side validation.

2. JavaScript Event Handling in Validation

Events are actions that occur in the page, and event handlers are functions that run in response. For validation:

- **submit:** fires when the form is submitted; the handler checks all fields and calls event.preventDefault() if any is invalid.
- **blur / change:** validates a field when the user leaves it.
- **input / keyup:** gives real-time feedback while typing (e.g. password strength).

Handlers are attached with addEventListener(), which keeps JavaScript separate from HTML.

3. Role of Regular Expressions

A regular expression (regex) describes a pattern of characters. It is used with test() or match() to check whether a string follows a required format.

- **Email:** /^[^\s@]+@[^\s@]+\.[^\s@]+$/ – text, then @, then a domain, a dot and an extension.
- **Phone (10-digit Indian mobile):** /^[6-9]\d{9}$/ – starts with 6–9 followed by nine digits.
- **Password:** /^(?=.*[A-Z])(?=.*\d).{8,}$/ – at least 8 characters, one capital and one digit.

4. Example

```
<form id="regForm" novalidate>
  <input type="text"   id="email" placeholder="Email">
  <span id="emailErr" style="color:red"></span><br>
  <input type="text"   id="phone" placeholder="Phone">
  <span id="phoneErr" style="color:red"></span><br>
  <button type="submit">Submit</button>
</form>

<script>
const emailRe = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const phoneRe = /^[6-9]\d{9}$/;

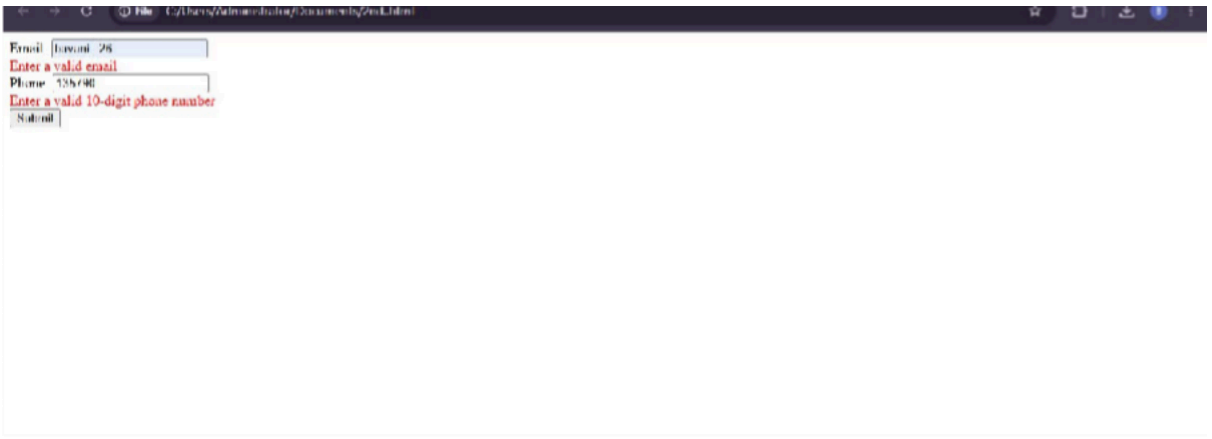
document.getElementById("regForm").addEventListener("submit", function(e){
  let valid = true;
  const email = document.getElementById("email").value.trim();
  const phone = document.getElementById("phone").value.trim();

  if(!emailRe.test(email)){
    emailErr.textContent = "Enter a valid email address"; valid = false;
  } else emailErr.textContent = "";
```

```
  if(!phoneRe.test(phone)){
    phoneErr.textContent = "Enter a valid 10-digit mobile number"; valid =
false;
  } else phoneErr.textContent = "";

  if(!valid) e.preventDefault();    // stop submission
});
</script>
```

5. Improvements for Usability and Security

- Show clear, specific error messages next to each field, and validate in real time (on blur/input).
- Use HTML5 attributes (required, type="email", pattern, minlength) and the correct input types so mobile keyboards adapt.
- Use ARIA attributes (aria-live, aria-invalid) for accessibility.
- Security: always repeat validation on the server because client-side checks can be bypassed; sanitise/escape input to prevent XSS and SQL injection; use HTTPS and CSRF tokens; never trust hidden fields.
- 

Question 3 (5 Marks)

1. CSS Box Model

Every HTML element is treated as a rectangular box. The box model describes the space an element occupies. Its components, from inside to outside, are:

- **Content:** the actual text or image (width and height).
- **Padding:** transparent space between the content and the border.
- **Border:** the line surrounding the padding and content.
- **Margin:** transparent space outside the border that separates the element from its neighbours.

Total width = content width + left/right padding + left/right border + left/right margin. With box-sizing:border-box, the declared width includes padding and border, which makes layouts predictable.

```
div{ width:300px; padding:20px; border:2px solid #000; margin:10px;
box-sizing:border-box; }
```

2. CSS Positioning and Its Effect on Layout

- **static (default):** follows the normal document flow; top/left have no effect.
- **relative:** offset from its normal position; the original space is still reserved.
- **absolute:** removed from the flow and positioned relative to the nearest positioned ancestor; other elements ignore it, so overlap can occur.
- **fixed:** removed from the flow and fixed to the viewport (e.g. sticky header); can cover content.
- **sticky:** behaves as relative until a scroll threshold, then sticks like fixed.

Wrong use of absolute/fixed positioning removes elements from the normal flow, which is a common reason why layouts break.

3. Possible Causes of Layout Breaking on Resize

- Fixed pixel widths and heights (e.g. width:1000px) that exceed small screens.
- Overuse of absolute/fixed positioning and floats without clearing.
- Missing viewport meta tag and missing box-sizing:border-box (padding adds to width).
- Images, tables or videos wider than their container (no max-width:100%).
- Fixed-height containers causing overflow, and no media queries or breakpoints.

4. Using Media Queries to Resolve the Problem

Media queries apply CSS rules only when conditions such as screen width are met, so the layout can be rearranged at defined breakpoints.

```
.wrapper{ width:90%; max-width:1100px; margin:auto; display:flex; gap:16px;
}
.sidebar{ width:30%; }
.content{ width:70%; }

@media (max-width:768px){
  .wrapper{ flex-direction:column; }
  .sidebar, .content{ width:100%; }
}
```

5. Best Practices for a Stable, Responsive Layout

- Use relative units (%, rem, fr, vw) and max-width instead of fixed widths.
- Apply * { box-sizing:border-box; } and a CSS reset.
- Prefer Flexbox/Grid over floats and heavy absolute positioning.
- Adopt a mobile-first approach with logical breakpoints (≈600 px, 768 px, 1024 px).
- Make media flexible (max-width:100%), use overflow-x:auto for wide tables, and test on multiple devices.



Question 4 (5 Marks)

1. Document Object Model (DOM) and Its Role

The DOM is a programming interface created by the browser when it loads a web page. It represents the HTML document as a tree of objects (nodes): the document is the root, followed by <html>, <head>, <body> and their child elements, attributes and text. JavaScript uses the DOM to access and change the content, structure and style of the page.

Role in web development: it connects HTML/CSS with JavaScript, enables dynamic and interactive pages, allows reaction to user events and lets pages be updated without a full reload.

2. Manipulating the DOM with JavaScript

- **Selecting:** getElementById(), querySelector(), querySelectorAll().
- **Changing content/attributes/styles:** textContent, innerHTML, setAttribute(), style, classList.
- **Creating and inserting:** createElement(), appendChild(), append(), insertBefore().
- **Removing:** remove(), removeChild().
- **Events:** addEventListener("click", handler) to trigger updates.

Because only the changed part of the tree is updated and re-rendered, the page does not reload.

3. Example – Adding Items Dynamically

```
<input type="text" id="task" placeholder="Enter task">
<button id="addBtn">Add</button>
<ul id="list"></ul>
<p id="count">Total tasks: 0</p>

<script>
const list  =
document.getElementById("list"); const count
= document.getElementById("count");

document.getElementById("addBtn").addEventListener("click",
  function(){ const input = document.getElementById("task");
  if(input.value.trim() === "") return;
  const li =
  document.createElement("li");          // safe: no HTML injection
  li.textContent = input.value;          // update DOM without
  list.appendChild(li);                  reload
  count.textContent = "Total tasks: " +
  list.children.length; input.value = "";
});
```

```
</script>
```

Each click adds a new list item and updates the count instantly, without reloading the page.

4. Improvements for Performance and User Experience

- Minimise DOM access: cache selected elements in variables and batch changes with DocumentFragment to reduce reflow/repaint.
- Use event delegation (one listener on the parent) instead of many listeners.
- Use textContent instead of innerHTML for user data to prevent XSS.
- Fetch data asynchronously using fetch() / AJAX and update only the required part.
- Debounce/throttle frequent events (scroll, input), and show loading indicators and aria-live messages for better UX and accessibility.

Question 5 (5 Marks)

To make the college website display properly on desktops, tablets and mobile phones, I would combine a flexible layout with CSS media queries.

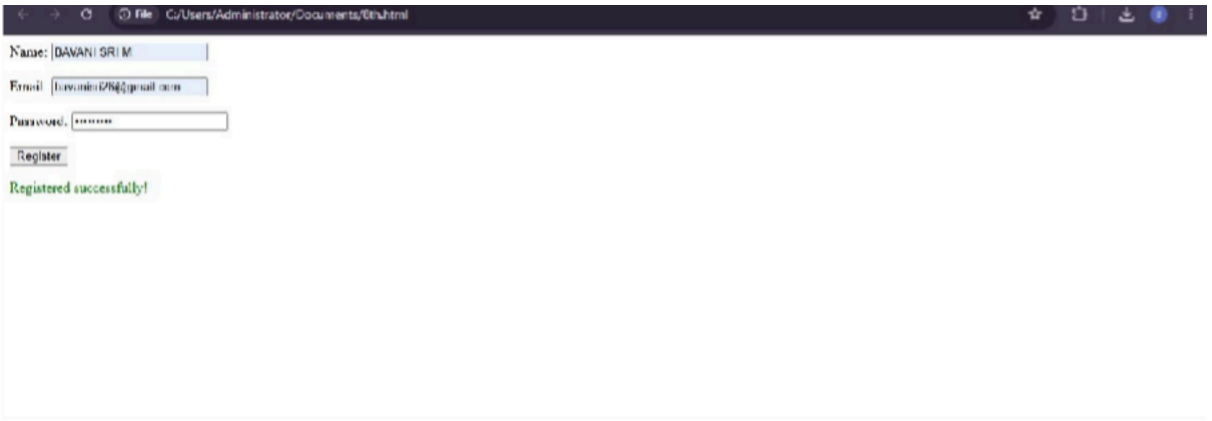
Steps

- **Viewport meta tag:** <meta name="viewport" content="width=device-width, initial-scale=1"> in the <head>.
- **Flexible layout:** use percentages/fr units, max-width containers, Flexbox or Grid, and max-width:100% for images.
- **Media queries:** define breakpoints – desktop (above 992 px: 3 columns), tablet (601–992 px: 2 columns) and mobile (up to 600 px: 1 column).
- Mobile-first approach with readable fonts and touch-friendly links.

```
.container{ display:flex; flex-wrap:wrap; gap:16px; max-width:1200px;
margin:auto; }
.box{ flex:1 1 30%; background:#eef2f7; padding:16px; }
img{ max-width:100%; height:auto; }

/* Tablet */
@media (max-width:992px){
  .box{ flex:1 1 45%; }
}
/* Mobile */
@media (max-width:600px){
  .box{ flex:1 1 100%; }
  nav ul{ flex-direction:column; }
}
```

Result: the departments, news and menu sections rearrange themselves according to screen width, giving a consistent and readable website on every device.



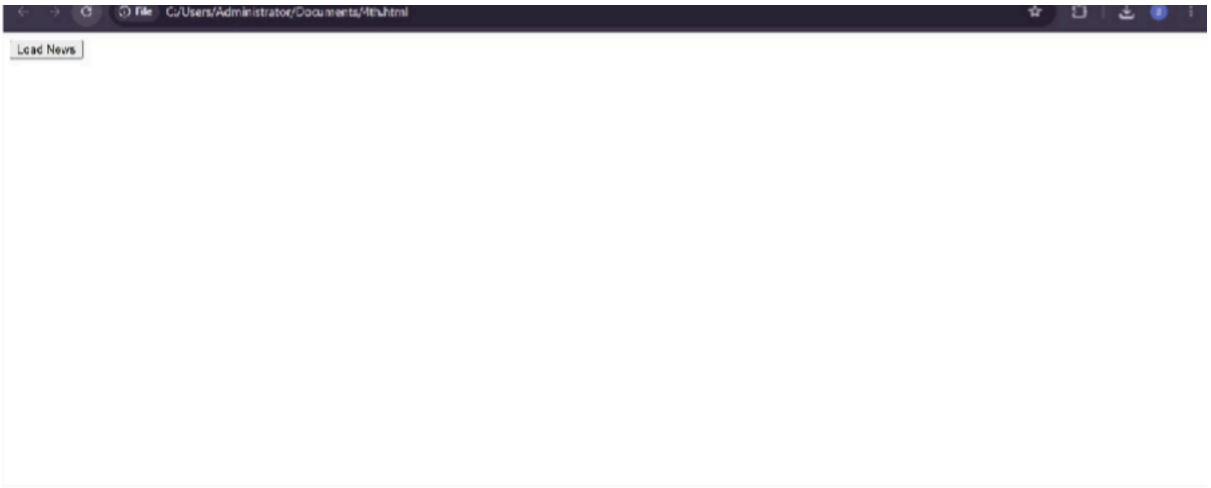
Question 6 (5 Marks)

I would style the plain product cards with a clean background colour, a rounded border, a soft box-shadow, proper padding/margin and a hover effect to make them attractive and clear.

- **Colours:** white card on a light-grey page, a contrasting colour for price and the "Add to Cart" button.
- **Border:** a thin light border with border-radius for rounded corners.
- **Shadow:** box-shadow to give depth; a stronger shadow on hover.
- **Spacing:** padding inside the card and margin/gap between cards.

```
body{ background:#f2f4f7; }
.product-card{
  background:#ffffff;
  border:1px solid
  #e0e0e0;
  border-radius:10px;
  padding:16px;
  margin:12px;
  width:240px;
  box-shadow:0 2px 6px
  rgba(0,0,0,0.15); text-align:center;
  transition:transform .3s, box-shadow .3s;
}
.product-card:hover{
  transform:translateY(-5px)
  ;
  box-shadow:0 8px 18px rgba(0,0,0,0.25);
}
.product-card img{ max-width:100%; border-radius:8px; }
.product-card .price{ color:#c62828; font-weight:bold; font-size:18px;
margin:8px 0; }
.product-card button{
  background:#2e7d32; color:#fff; border:none;
  padding:10px 16px; border-radius:6px; cursor:pointer;
}
.product-card button:hover{ background:#1b5e20; }
```

This gives the cards a modern appearance, separates them clearly from the background and improves readability and attention to the price and button.



Question 7 (5 Marks)

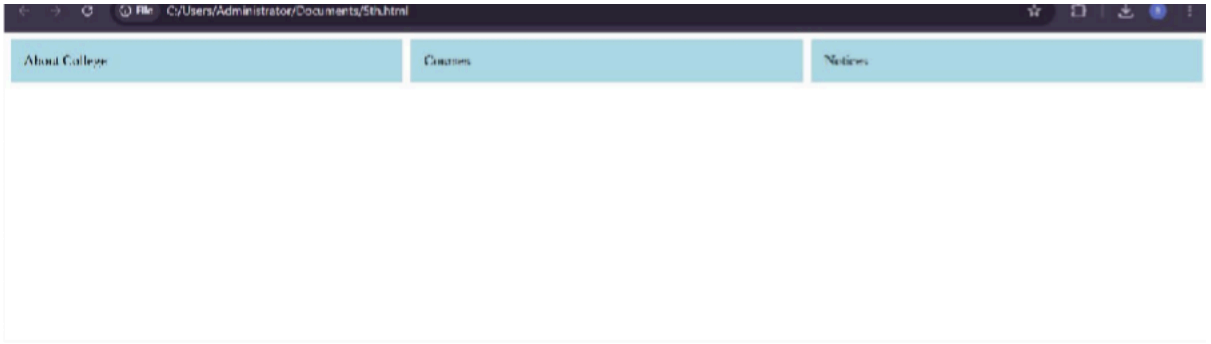
Overlapping usually happens because the menu has a fixed width, uses position:absolute/fixed (so it leaves the normal flow) or has items that do not wrap. I would correct it with the following layout techniques:

- Keep the menu in the normal flow (position:relative/static) or use position:sticky with a proper z-index and spacing.
- Use Flexbox with flex-wrap, or switch to flex-direction:column on small screens.
- Use a media query to convert the menu into a vertical or collapsible (hamburger) menu on mobile.
- Use relative units and box-sizing:border-box so the menu never exceeds the screen width.

```
nav ul{ display:flex; flex-wrap:wrap; list-style:none; gap:10px;
} nav li a{ display:block; padding:10px 14px; }

@media (max-width:600px){
  nav ul{ flex-direction:column; width:100%; }  /* stack the items
  */ nav li a{ border-bottom:1px solid #ccc; }
}
```

Because the menu now stays in the document flow and stacks vertically, the content below it is pushed down rather than being covered.



Question 8 (5 Marks)

I would attach a submit event listener to the form, read each field value, check whether it is empty, display an error message and stop the submission with preventDefault() if any field is invalid.

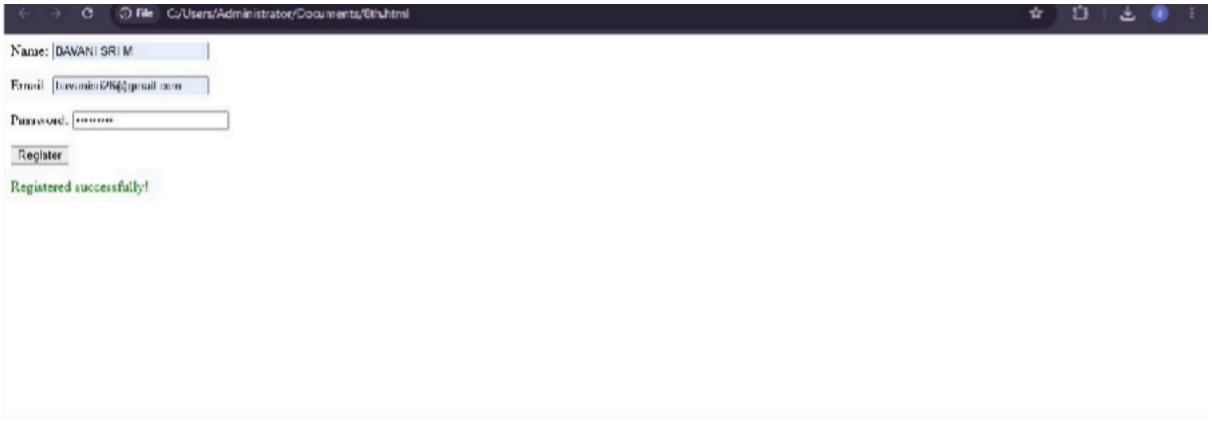
```
<form id="regForm">
  <input type="text" id="name" placeholder="Name"><span id="e1"></span><br>
  <input type="text" id="email" placeholder="Email"><span
  id="e2"></span><br>
  <input type="password" id="pwd"
placeholder="Password"><span id="e3"></span><br>
  <button type="submit">Register</button>
</form>

<script>
document.getElementById("regForm").addEventListener("submit", function(e){
  let ok = true;
  const name= document.getElementById("name").value.trim();
  const email =
  document.getElementById("email").value.trim(); const pwd
  = document.getElementById("pwd").value;

  if(name === ""){  e1.textContent = " Name is required"; ok = false;
  } if(email === ""){ e2.textContent = " Email is required";  }ok = false;
  } if(pwd === ""){      e3.textContent = " Password is required"; ok =
  false; }

  if(!ok){ e.preventDefault(); }     // block
  submission else  { alert("Registration
  successful"); }
});
</script>
```

The form is submitted only when no field is empty. For better validation, the required attribute and regular expressions can be added, and server-side validation must also be done.



Question 9 (5 Marks)

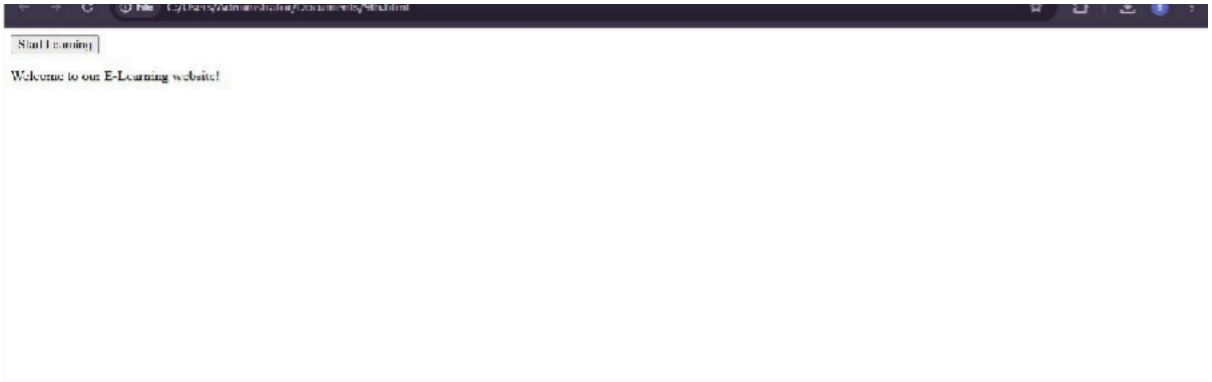
I would create a button in HTML, then use JavaScript to select it and attach a click event handler that displays the welcome message (in an alert or inside the page).

```
<button id="welcomeBtn">Click Me</button>
<p id="msg"></p>

<script>
```

```
document.getElementById("welcomeBtn").addEventListener("click", function(){
  document.getElementById("msg").textContent =
        "Welcome to our E-Learning Website!";
  // or: alert("Welcome to our E-Learning Website!");
});
</script>
```

Working: when the user clicks the button, the click event fires, the function runs and the paragraph with id "msg" is updated with the welcome message. Using addEventListener() keeps HTML and JavaScript separate; the older alternative is the inline onclick attribute.



Question 10 (5 Marks)

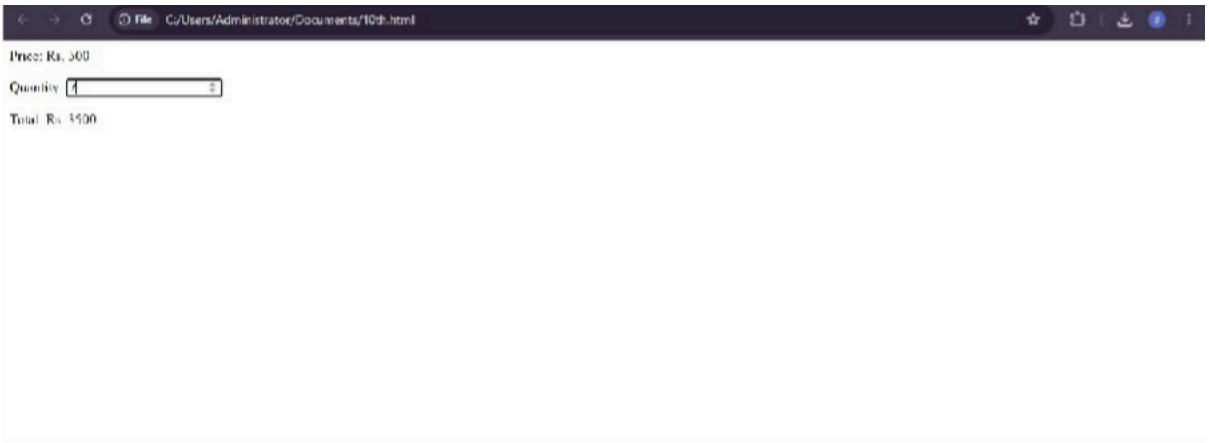
I would use an input of type number for the quantity and listen for its input/change event. The handler reads the quantity, multiplies it by the unit price and writes the result into the page using the DOM.

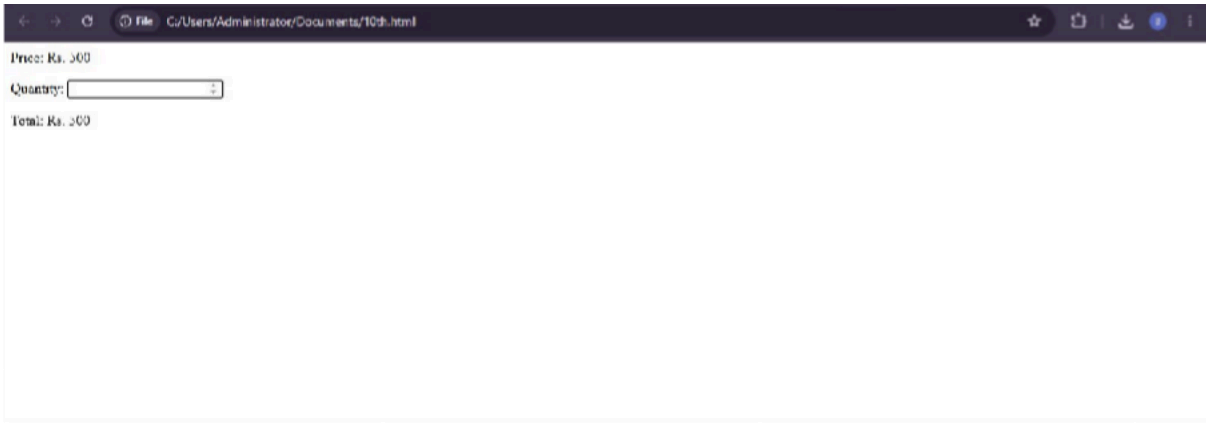
```
<p>Price per item: &#8377;<span id="price">500</span></p>
<label>Quantity:
  <input type="number" id="qty" value="1" min="1">
</label>
<p>Total Price: &#8377;<span id="total">500</span></p>

<script>
const unitPrice =
Number(document.getElementById("price").textContent); const qtyBox =
document.getElementById("qty");
const totalSpan = document.getElementById("total");

qtyBox.addEventListener("input", function(){
  let qty = parseInt(qtyBox.value);
  if(isNaN(qty) || qty < 1){ qty = 1; }   // validation
  totalSpan.textContent = (unitPrice * qty).toFixed(2);
});
</script>
```

Working: each time the quantity changes, the function recalculates total = price × quantity and updates the displayed total immediately without reloading the page. The price should also be re-verified on the server before payment.


File C:/Users/Administrator/Documents/10th.html
Price: Rs. 500
Quantity: 1
Total: Rs. 3500


File C:/Users/Administrator/Documents/10th.html
Price: Rs. 500
Quantity:
Total: Rs. 500